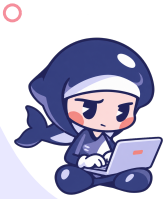
Module 1: Transcriptomic Landscape

CONTENTS PAGE

01	pipeline/scripts/rna/pileup_candidate_base_counts.py	2
02	pipeline/scripts/rna/export_alt20_depth100.py	8
03	pipeline/scripts/rna/export_positive_strand.py	10
04	pipeline/config/current_hek293t.json	12
05	pipeline/env/rna_counting.yml	13
06	pipeline/env/genomic_catalogue.yml	14
07	pipeline/env/reditools2_py2.yml	15





pipeline/scripts/rna/pileup_candidate_base_counts.py

```python
#!/usr/bin/env python3
"""Measure quality-filtered A/C/G/T counts at candidate sites in every sample.

Unlike a caller table, this script records counts even when the alternate allele does
not cross a calling threshold. This is required for continuous editing-efficiency
labels and for distinguishing a true zero from a control non-call.
"""

from __future__ import annotations

import argparse
import csv
import gzip
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Sequence

BASES = ("A", "C", "G", "T")


def open_text(path: Path | str, mode: str = "rt"):
    path = str(path)
    if path.endswith(".gz"):
        return gzip.open(path, mode, newline="")
    return open(path, mode, newline="")


def norm_chrom(chrom: str) -> str:
    chrom = chrom.strip()
    return chrom[3:] if chrom.lower().startswith("chr") else chrom


def find_column(fieldnames: Sequence[str], candidates: Sequence[str], label: str) -> str:
    lookup = {name.lower(): name for name in fieldnames}
    for candidate in candidates:
        if candidate.lower() in lookup:
            return lookup[candidate.lower()]
    raise SystemExit(
        f"ERROR: cannot find {label} column. Available columns: {', '.join(fieldnames)}"
    )


@dataclass(frozen=True, order=True)
class Site:
    chrom: str
    position: int
    ref: str
    alt: str


def read_manifest(path: Path) -> List[dict]:
    with open(path, newline="") as handle:
        rows = list(csv.DictReader(handle, delimiter="\t"))
    if not rows or not {"sample", "group", "replicate"}.issubset(rows[0]):
        raise SystemExit(
            "ERROR: manifest must contain sample, group and replicate columns"
```





```
        )
    return rows


def read_sites(path: Path) -> List[Site]:
    with open_text(path) as handle:
        reader = csv.DictReader(handle, delimiter="\t")
        if reader.fieldnames is None:
            raise SystemExit(f"ERROR: candidate table has no header: {path}")
        chrom_col = find_column(
            reader.fieldnames, ["chrom", "chr", "chromosome", "region"], "chromosome"
        )
        pos_col = find_column(
            reader.fieldnames, ["position", "pos", "coordinate"], "position"
        )
        ref_col = find_column(reader.fieldnames, ["ref", "reference"], "reference allele")
        alt_col = find_column(
            reader.fieldnames, ["alt", "alternate", "alternative"], "alternate allele"
        )
        sites = {
            Site(
                chrom=row[chrom_col].strip(),
                position=int(row[pos_col]),
                ref=row[ref_col].upper(),
                alt=row[alt_col].upper(),
            )
            for row in reader
            if row.get(chrom_col)
            and row.get(pos_col)
            and row.get(ref_col)
            and row.get(alt_col)
        }
    return sorted(sites, key=lambda s: (norm_chrom(s.chrom), s.position, s.ref, s.alt))


def resolve_bam_path(bam_dir: Path, sample: str, srr: str) -> Path:
    candidates = [
        bam_dir / f"{sample}.splitncigarreads.bam",
        bam_dir / f"{srr}.splitncigarreads.bam" if srr else Path("/__missing__"),
    ]
    for path in candidates:
        if path.is_file():
            return path
    checked = ", ".join(str(p) for p in candidates if str(p) != "/__missing__")
    raise SystemExit(f"ERROR: BAM not found for {sample}. Checked: {checked}")


def reference_name_map(references: Sequence[str]) -> Dict[str, str]:
    mapping: Dict[str, str] = {}
    for reference in references:
        key = norm_chrom(reference)
        if key in mapping and mapping[key] != reference:
            raise SystemExit(
                f"ERROR: ambiguous BAM contig names after chr normalization: {mapping[key]} and {r
eference}"
            )
        mapping[key] = reference
    return mapping


def count_site(
```





```
    alignment,
    bam_chrom: str,
    position: int,
    min_mapq: int,
    min_baseq: int,
    max_depth: int,
    require_nh_unique: bool = True,
) -> dict:
    counts = {base: 0 for base in BASES}
    forward = {base: 0 for base in BASES}
    reverse = {base: 0 for base in BASES}
    excluded = {
        "low_mapq": 0,
        "low_baseq": 0,
        "duplicate": 0,
        "secondary": 0,
        "supplementary": 0,
        "qcfail": 0,
        "deletion_or_refskip": 0,
        "non_acgt": 0,
        "missing_nh": 0,
        "nh_multimapper": 0,
    }

    for column in alignment.pileup(
        bam_chrom,
        position - 1,
        position,
        truncate=True,
        stepper="all",
        min_base_quality=0,
        min_mapping_quality=0,
        max_depth=max_depth,
    ):
        if column.reference_pos != position - 1:
            continue
        for pileup_read in column.pileups:
            read = pileup_read.alignment
            if read.is_unmapped:
                continue
            if read.is_duplicate:
                excluded["duplicate"] += 1
                continue
            if read.is_secondary:
                excluded["secondary"] += 1
                continue
            if read.is_supplementary:
                excluded["supplementary"] += 1
                continue
            if read.is_qcfail:
                excluded["qcfail"] += 1
                continue
            if read.mapping_quality < min_mapq:
                excluded["low_mapq"] += 1
                continue
            if require_nh_unique:
                if not read.has_tag("NH"):
                    excluded["missing_nh"] += 1
                    continue
                if int(read.get_tag("NH")) != 1:
                    excluded["nh_multimapper"] += 1
```





```
                    continue
            if (
                pileup_read.is_del
                or pileup_read.is_refskip
                or pileup_read.query_position is None
            ):
                excluded["deletion_or_refskip"] += 1
                continue
            query_position = pileup_read.query_position
            qualities = read.query_qualities
            if qualities is None or qualities[query_position] < min_baseq:
                excluded["low_baseq"] += 1
                continue
            sequence = read.query_sequence
            if sequence is None:
                excluded["non_acgt"] += 1
                continue
            base = sequence[query_position].upper()
            if base not in counts:
                excluded["non_acgt"] += 1
                continue
            counts[base] += 1
            (reverse if read.is_reverse else forward)[base] += 1

    result = {}
    for base in BASES:
        result[f"{base}_count"] = counts[base]
        result[f"{base}_forward_count"] = forward[base]
        result[f"{base}_reverse_count"] = reverse[base]
    result["acgt_depth"] = sum(counts.values())
    result["excluded_read_observations"] = sum(excluded.values())
    for reason, value in excluded.items():
        result[f"excluded_{reason}"] = value
    return result


def output_fields() -> List[str]:
    fields = ["sample", "group", "replicate", "chrom", "position", "ref", "alt"]
    for base in BASES:
        fields += [f"{base}_count", f"{base}_forward_count", f"{base}_reverse_count"]
    fields += [
        "acgt_depth",
        "ref_count",
        "alt_count",
        "allele_depth",
        "edit_rate",
        "alt_forward_count",
        "alt_reverse_count",
        "alt_strand_balance",
        "excluded_read_observations",
        "excluded_low_mapq",
        "excluded_low_baseq",
        "excluded_duplicate",
        "excluded_secondary",
        "excluded_supplementary",
        "excluded_qcfail",
        "excluded_deletion_or_refskip",
        "excluded_non_acgt",
        "excluded_missing_nh",
        "excluded_nh_multimapper",
    ]
```





```
    return fields


def main() -> None:
    parser = argparse.ArgumentParser(
        description=(
            "Extract quality-filtered A/C/G/T counts at every candidate site from all BAMs. "
            "Counts are emitted even for sites absent from a variant-caller output."
        )
    )
    parser.add_argument("--manifest", required=True, type=Path)
    parser.add_argument("--candidate-table", required=True, type=Path)
    parser.add_argument("--bam-dir", required=True, type=Path)
    parser.add_argument("--output-dir", required=True, type=Path)
    parser.add_argument(
        "--reference",
        type=Path,
        default=None,
        help="Reference FASTA; required for CRAM input",
    )
    parser.add_argument("--min-mapq", type=int, default=30)
    parser.add_argument("--min-baseq", type=int, default=20)
    parser.add_argument("--max-depth", type=int, default=1_000_000)
    parser.add_argument(
        "--allow-missing-nh",
        action="store_true",
        help="Legacy opt-out. Strict mode requires an NH tag equal to 1 for every counted read.",
    )
    args = parser.parse_args()

    if args.min_mapq < 0 or args.min_baseq < 0 or args.max_depth < 1:
        parser.error(
            "quality thresholds must be non-negative and max-depth must be positive"
        )

    try:
        import pysam
    except ImportError as exc:
        raise SystemExit(
            "ERROR: pysam is required. Create the environment with pipeline/env/rna_counting.yml"
        ) from exc

    manifest = read_manifest(args.manifest)
    sites = read_sites(args.candidate_table)
    if not sites:
        raise SystemExit("ERROR: candidate table contains no sites")

    args.output_dir.mkdir(parents=True, exist_ok=True)
    fields = output_fields()
    reference_filename = str(args.reference) if args.reference else None

    for meta in manifest:
        sample = meta["sample"]
        bam_path = resolve_bam_path(args.bam_dir, sample, meta.get("srr", ""))
        output_path = args.output_dir / f"{sample}.candidate_base_counts.tsv.gz"
        mode = "rc" if bam_path.suffix.lower() == ".cram" else "rb"
        with pysam.AlignmentFile(
            bam_path, mode, reference_filename=reference_filename
        ) as alignment, open_text(output_path, "wt") as output:
            chrom_map = reference_name_map(alignment.references)
            writer = csv.DictWriter(output, fieldnames=fields, delimiter="\t")
```





```
            writer.writeheader()
            for site in sites:
                bam_chrom = chrom_map.get(norm_chrom(site.chrom))
                if bam_chrom is None:
                    raise SystemExit(
                        f"ERROR: candidate contig {site.chrom!r} is absent from BAM {bam_path}"
                    )
                measured = count_site(
                    alignment,
                    bam_chrom,
                    site.position,
                    args.min_mapq,
                    args.min_baseq,
                    args.max_depth,
                    require_nh_unique=not args.allow_missing_nh,
                )
                ref_count = measured.get(f"{site.ref}_count", 0)
                alt_count = measured.get(f"{site.alt}_count", 0)
                allele_depth = ref_count + alt_count
                alt_forward = measured.get(f"{site.alt}_forward_count", 0)
                alt_reverse = measured.get(f"{site.alt}_reverse_count", 0)
                row = {
                    "sample": sample,
                    "group": meta["group"],
                    "replicate": meta["replicate"],
                    "chrom": site.chrom,
                    "position": site.position,
                    "ref": site.ref,
                    "alt": site.alt,
                    **measured,
                    "ref_count": ref_count,
                    "alt_count": alt_count,
                    "allele_depth": allele_depth,
                    "edit_rate": (
                        f"{alt_count / allele_depth:.10g}" if allele_depth else "NA"
                    ),
                    "alt_forward_count": alt_forward,
                    "alt_reverse_count": alt_reverse,
                    "alt_strand_balance": (
                        f"{min(alt_forward, alt_reverse) / max(alt_forward, alt_reverse):.10g}"
                        if max(alt_forward, alt_reverse) > 0
                        else "NA"
                    ),
                }
                writer.writerow(row)
        print(f"candidate base counts: {sample}\t{len(sites)}\t{output_path}")


if __name__ == "__main__":
    main()
```





pipeline/scripts/rna/export_alt20_depth100.py

```
"""Final subset export from ALREADY background/SNP-filtered positives."""

import argparse
import csv
import math
import statistics
from collections import defaultdict
from pathlib import Path

KEY = ('chromosome', 'position', 'ref', 'alt')


def read(path):
    with open(path, newline='') as f:
        reader = csv.DictReader(f, delimiter='\t')
        return reader.fieldnames, list(reader)


def select(sites, counts, samples):
    if len(samples) != 4 or len(set(samples)) != 4:
        raise ValueError('Exactly four distinct biological replicates required')
    keys = [tuple(r[k] for k in KEY) for r in sites]
    if len(set(keys)) != len(keys):
        raise ValueError('Duplicate positive site')
    evidence = defaultdict(dict)
    for row in counts:
        key = tuple(row[k] for k in KEY)
        sample = row['sample']
        if sample not in samples:
            raise ValueError('Unexpected sample: ' + sample)
        if sample in evidence[key]:
            raise ValueError('Duplicate site/sample count')
        depth, alt = int(row['depth']), int(row['alt_reads'])
        if not 0 <= alt <= depth or depth <= 0:
            raise ValueError('Invalid allele counts')
        evidence[key][sample] = (depth, alt)
    selected = []
    for site, key in zip(sites, keys):
        records = evidence[key]
        if set(records) != set(samples):
            raise ValueError('Missing replicate evidence: ' + str(key))
        if not all(d >= 100 and a >= 20 and a / d >= 0.005 for d, a in records.values()):
            continue
        row = dict(site)
        rates = [records[s][1] / records[s][0] for s in samples]
        rate = float(site['editing_rate'])
        if not math.isfinite(rate) or abs(rate - statistics.median(rates)) > 1e-8:
            raise ValueError('Site rate is not median of four replicate ALT/depth rates')
        for sample in samples:
            d, a = records[sample]
            row.update(
                {
                    sample + '_depth': d,
                    sample + '_alt_reads': a,
                    sample + '_editing_rate': a / d,
                }
```





```
            )
        selected.append(row)
    return selected


def main():
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument('--positive', required=True)
    p.add_argument('--counts', required=True)
    p.add_argument('--samples', nargs=4, required=True)
    p.add_argument('--output', type=Path, required=True)
    args = p.parse_args()
    fields, sites = read(args.positive)
    _, counts = read(args.counts)
    selected = select(sites, counts, args.samples)
    fields = fields + [
        s + '_' + k for s in args.samples for k in ('depth', 'alt_reads', 'editing_rate')
    ]
    with args.output.open('x', newline='') as f:
        w = csv.DictWriter(f, fields, delimiter='\t', lineterminator='\n')
        w.writeheader()
        w.writerows(selected)
    print(f'{len(sites)} background-filtered positives -> {len(selected)} final sites')


if __name__ == '__main__':
    main()
```





pipeline/scripts/rna/export_positive_strand.py

```
"""Add transcript/editing strand to already strand-compatible C-to-U positives.

This is an annotation-only export, not a caller or a strand inference tool for
arbitrary variants. Inputs must already have passed transcript-aware annotation.
Genomic alleles and pre-oriented sequence contexts are preserved verbatim.
"""

import argparse
import csv
from collections import Counter
from pathlib import Path

KEY = ('chromosome', 'position', 'ref', 'alt')
STRANDS = {('C', 'T'): '+', ('G', 'A'): '-'}


def annotate(fields, rows):
    if not fields or len(fields) != len(set(fields)):
        raise ValueError('Missing or duplicate column names')
    if not set(KEY).issubset(fields):
        raise ValueError('Required columns: ' + ', '.join(KEY))
    output_fields = list(fields)
    if 'strand' not in fields:
        output_fields.insert(output_fields.index('position') + 1, 'strand')
    result, seen = [], set()
    for line, original in enumerate(rows, start=2):
        if set(original) != set(fields) or any(v is None for v in original.values()):
            raise ValueError(f'Line {line}: malformed TSV row')
        if (
            not original['chromosome']
            or not original['position'].isdigit()
            or int(original['position']) < 1
        ):
            raise ValueError(
                f'Line {line}: expected a nonempty chromosome and positive 1-based position'
            )
        key = tuple(original[k] for k in KEY)
        if key in seen:
            raise ValueError(f'Line {line}: duplicate site {key}')
        seen.add(key)
        strand = STRANDS.get((original['ref'], original['alt']))
        if strand is None:
            raise ValueError(f'Line {line}: expected genomic C>T or G>A, got {key}')
        if 'strand' in fields and original['strand'] != strand:
            raise ValueError(
                f'Line {line}: existing strand conflicts with the C-to-U convention'
            )
        if 'sequence_101nt' in fields:
            seq = original['sequence_101nt']
            if len(seq) != 101 or seq[50] != 'C' or set(seq) - set('ACGT'):
                raise ValueError(
                    f'Line {line}: expected an already oriented unambiguous 101-nt C-centered sequ
ence'
                )
        result.append({**original, 'strand': strand})
    return output_fields, result
```





```


def export(source, destination):
    with Path(source).open(newline='', encoding='utf-8') as handle:
        reader = csv.DictReader(handle, delimiter='\t')
        fields, rows = annotate(reader.fieldnames, list(reader))
    # Validate the entire input before creating any output. Never replace files.
    with Path(destination).open('x', newline='', encoding='utf-8') as handle:
        writer = csv.DictWriter(
            handle, fieldnames=fields, delimiter='\t', lineterminator='\n'
        )
        writer.writeheader()
        writer.writerows(rows)
    return Counter(row['strand'] for row in rows)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--positive', required=True, type=Path)
    parser.add_argument('--output', required=True, type=Path)
    args = parser.parse_args()
    counts = export(args.positive, args.output)
    print(f"Exported {sum(counts.values())} sites: +={counts['+']}, -={counts['-']}")


if __name__ == '__main__':
    main()
```





pipeline/config/current_hek293t.json

```
{
  "schema_version": 1,
  "species": "Homo sapiens",
  "assembly": "GRCh38",
  "gtf": "GENCODE v50 primary assembly",
  "replicates_per_group": 4,
  "priority_callable_samples": 16,
  "comparison_callable_samples": 36,
  "min_baseq": 31,
  "min_mapq": 31,
  "require_nh": 1,
  "calling": {"me": 5, "men": 1},
  "initial_positive": {"formal_replicates": 4, "min_alt_each": 5, "min_af_each": 0.005, "min_depth
_all_included": 20, "max_treated_mad": 0.05},
  "background": {"groups": ["Control", "APOBEC-only", "PUF-only"], "max_group_median": 0.02, "max_
group_mad": 0.02, "max_formal_calls_per_group": 0, "max_editor_only_minus_control": 0.02},
  "fdr": {"max": 0.05, "method": "BH of maximum one-sided pooled-read Fisher p across three contro
ls per site"},
  "final_export": {"min_alt_each": 20, "min_depth_each": 100, "replicates_required": 4},
  "rate": "median of four ALT/(REF+ALT) replicate rates",
  "min_increase": null,
  "negative_generation": false,
  "display_alias": {"GVD": "GVE"},
  "positive_strand": {"precondition": "upstream transcript-strand-compatible C-to-U positives", "C
>T": "+", "G>A": "-", "coordinates": "genomic, 1-based", "sequence": "already transcript-oriented;
 C at zero-based index 50"},
  "canonical_contigs": {"autosomes": "1-22", "sex": ["X", "Y"], "mitochondrial_aliases": ["M", "MT
"], "optional_prefix": "chr"},
  "status": "deployed six-group computation and figures completed 2026-09-24; not biological valid
ation"
}
```





pipeline/env/rna_counting.yml

```yaml
name: rewire_rna_counting
channels:
  - conda-forge
  - bioconda
dependencies:
  # Supported repository environment, not the frozen production interpreter.
  - python=3.12
  - pysam=0.24.*
  - samtools=1.23.*

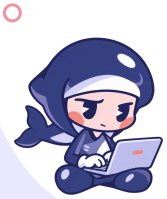
```





pipeline/env/genomic_catalogue.yml

```yaml
name: rewire_catalogue
channels:
  - conda-forge
  - bioconda
  - defaults
dependencies:
  - python=3.11
  - crossmap
  - bcftools
  - samtools
  - htslib
  - curl
```





pipeline/env/reditools2_py2.yml

```
name: reditools2_py2
channels:
  - defaults
dependencies:
  - python=2.7.18
```

